

ELECTROSPHERE

AI-Based Road Accident Detection System

Technical Project Closure Report

Team: Sinchana S & Gagandeep C

Department: Electronics and Communication Engineering

Repository: [gagan23-ai/highway-accident-detection](#)

Report scope: Work completed from problem definition through algorithm development, design exploration, automation, evaluation and web-interface integration.

1. Problem Statement

Road accidents are a major safety concern, while continuous manual monitoring of road-surveillance footage is difficult and time-consuming. The objective of ElectroSphere was to develop a computer-vision-based system that analyses road-traffic video and identifies possible accident events automatically.

The project focuses on detecting vehicle interactions that are consistent with a collision and providing useful evidence such as the first detected accident timestamp, confidence/event score and an accident snapshot. The final implementation combines lightweight object detection, multi-object tracking and temporal vehicle-interaction analysis.

2. Understanding Requirements

- Detect relevant road vehicles from traffic videos.
- Maintain vehicle identity across analysed frames so that motion can be evaluated over time.
- Identify possible collision events without treating every close vehicle pair as an accident.
- Provide an approximate impact timestamp and visual evidence.
- Evaluate the detector on accident and non-accident videos.
- Automate testing and error analysis to support quality improvement.
- Provide a web interface for video upload and result visualization.
- Keep the detection pipeline separate from the web layer so that the detector can run independently.
- Maintain source code and project documentation in the GitHub repository.

3. Design Space Exploration

3.1 Direct video classification

A direct accident/non-accident video classifier is possible, but it requires a sufficiently large and diverse labelled dataset and provides less explicit information about the physical interaction that triggered a prediction. It was therefore not selected as the primary approach.

3.2 Object detection followed by accident-event analysis

The selected design first detects vehicles and then analyses their spatial and motion relationships. This makes the decision process more interpretable and allows individual signals such as approach, contact, speed change and direction change to be examined.

3.3 Model size selection

YOLO11n was selected as the vehicle detector because it provides a lightweight detection model suitable for the available CPU-based development environment. The repository configures 640×640 input, 15 FPS target sampling, 0.25 detection confidence and 0.45 detector IoU threshold.

3.4 Tracking and temporal analysis

A tracking-based approach was selected instead of treating every frame independently. ByteTrack is used through Ultralytics tracking to assign vehicle IDs. These IDs allow trajectories to be built and motion changes to be analysed over a short temporal window.

4. Trade-Off Analysis: FPS, Input Size and Model Complexity

Parameter	Selected setting	Reason
Analysis FPS	15 FPS	Reduces computation while retaining sufficient temporal information for event analysis

Parameter	Selected setting	Reason
YOLO input size	640 × 640	Balances spatial detail and CPU processing cost.
Detector	YOLO11n	Lightweight model suited to the available hardware.
Tracker	ByteTrack	Maintains vehicle identities for trajectory-based analysis.

The design prioritizes a practical balance between processing cost and temporal information rather than maximizing model size. The repository implementation uses CPU-compatible inference in the project development environment.

5. Algorithm Pipeline

The implemented pipeline is: Video → frame sampling at target FPS → 640×640 preprocessing → YOLO11n vehicle detection → ByteTrack tracking → vehicle trajectories → pairwise vehicle analysis → approach/contact/motion/persistence evidence → event score and independent-signal check → accident decision → impact timestamp/confidence/snapshot.

- Vehicle classes used are car, motorcycle, bus and truck.
- Bounding-box centres and dimensions are used for geometric analysis.
- Vehicle-pair distance and bounding-box gap/IoU are evaluated.
- Approach behaviour and closing speed are evaluated before contact.
- Speed and direction changes are evaluated as abnormal-motion evidence.
- Post-contact persistence is checked over a confirmation window.
- Normal close traffic with smooth motion receives a penalty to reduce false positives.
- An event requires a minimum score of 5 and at least 2 independent evidence groups.

6. Accident-Event Logic and Key Parameters

Parameter	Value	Purpose
TARGET_FPS	15.0	Temporal sampling rate
YOLO_SIZE	640	Detector input dimension
YOLO_CONF	0.25	Minimum detector confidence
YOLO_IOU	0.45	Detector NMS IoU setting
MAX_PAIR_DISTANCE	135	Limits relevant vehicle-pair distance
CONTACT_GAP	18	Near-contact tolerance
CONTACT_IOU	0.02	Vehicle overlap/contact signal
MIN_CLOSING_SPEED	1.5	Approach evidence threshold
MIN_SPEED_CHANGE	4.0	Abnormal speed-change threshold
MIN_DIRECTION_CHANGE_DEG	28°	Abnormal direction-change threshold
CONFIRMATION_WINDOW	7	Post-contact confirmation window
MIN_EVENT_SCORE	5	Minimum event score
MIN_INDEPENDENT_SIGNALS	2	Minimum independent evidence groups

The accident confidence produced by the project is a heuristic event confidence derived from evidence; it is not a calibrated probability and should not be confused with YOLO object-detection confidence.

7. Automation and Quality-Improvement Experiments

The project includes an automated video-analysis script used to process videos consistently and generate detection records. The implementation stores vehicle IDs, timestamps, classes, detection confidence and bounding-box information, enabling trajectory and pairwise event analysis.

- Automated frame sampling and video processing.
- Persistent vehicle tracking through ByteTrack.
- Trajectory construction for tracked vehicles.
- Automatic pair-track construction for relevant vehicle pairs.
- Automatic evaluation of approach, contact, motion and persistence evidence.
- Normal-traffic suppression based on sustained proximity and smooth trajectories.
- Automatic selection of the strongest candidate event rather than choosing an event solely from minimum vehicle distance.
- Impact timestamp is associated with the transition into contact, rather than simply the frame with the smallest distance.

These experiments were used to reduce false positives from normal dense traffic and to avoid weak single-signal accident decisions.

8. Final Results

Metric	Result
Accuracy	80.43%
Precision	88.24%
Recall	78.95%
F1 Score	83.33%

The reported results summarize the project evaluation and should be interpreted as performance of the complete accident-detection pipeline on the tested dataset, not as the standalone accuracy of YOLO11n.

Interpretation

The precision of 88.24% indicates that a high proportion of detected accident events were correct. The recall of 78.95% indicates that some genuine accident cases were still missed. The F1 score of 83.33% provides a combined view of precision and recall, while the overall accuracy is 80.43%.

9. Web Interface Development and Deployment

The GitHub repository contains a Flask-based web interface. The interface accepts a video upload, allows the uploaded video to be previewed, calls the accident-detection pipeline through `detector_adapter.py` and displays the resulting accident/no-accident status.

- Video upload and browser playback.
- Detection request through the web interface.
- Accident/no-accident result.
- Confidence display when returned by the detector.
- First accident timestamp.
- Accident snapshot when supplied by the detector.
- Clickable timestamp to move the result video to the detected event.

The repository README explicitly keeps the web interface separate from the core accident-detection algorithm. The adapter layer passes the video path to the detector and returns accident, confidence, timestamp and snapshot information.

Deployment

The repository documents local Flask deployment using the project Python environment, requirements installation and python app.py, with the interface served locally at <http://127.0.0.1:5000>. The current repository evidence supports local deployment/demo; a production cloud deployment is not claimed here.

10. SMS Interface

SMS alerting was identified as part of the intended accident-response functionality. However, the current GitHub repository reviewed for this closure report does not contain an SMS implementation in app.py or the README. Therefore, the SMS component should be reported as an integration/planned feature unless the team has a separate implementation that is being demonstrated and can be added to the repository.

The intended workflow is: Accident detected → event confirmed → alert generated → SMS notification to a configured recipient.

11. Clean Snapshots for the Final Report

- Web interface showing video upload/preview.
- Detection result showing vehicle/event annotations.
- Accident result page showing first timestamp and confidence.
- Accident snapshot showing the detected event clearly.
- Result video view with the timestamp navigation control.
- SMS screenshot only if the SMS integration is actually implemented and demonstrated.

Screenshots should be cropped cleanly, should show the relevant result without desktop clutter, and should use consistent captions such as Figure 1, Figure 2, etc.

12. GitHub Repository and Folder Structure

Repository: [gagan23-ai/highway-accident-detection](https://github.com/gagan23-ai/highway-accident-detection)

Path	Purpose
app.py	Flask web application and web-layer control.
detector_adapter.py	Adapter between the web interface and the team's detection pipeline.
stage1_100_video_automation.py	Video processing, tracking, trajectory/event analysis and automated detection logic.
templates/	HTML templates for the web interface.
static/	Static web assets such as CSS.
requirements.txt	Python dependencies required by the project.
README.md	Repository documentation, setup, usage and interface integration details.
.gitignore	Git ignore configuration.

The repository README should be treated as the detailed repository-level documentation. This technical report provides the closure summary and engineering decisions, while the README contains the setup and integration details.

13. Conclusion

ElectroSphere developed a practical computer-vision pipeline for road accident detection by combining YOLO11n vehicle detection, ByteTrack tracking and temporal vehicle-interaction analysis. The system goes beyond simple vehicle proximity by evaluating approach behaviour, contact, abnormal speed/direction changes and post-contact persistence, while also suppressing evidence from normal smooth close traffic.

The project progressed from requirement understanding and design-space exploration to model/tracker selection, trade-off analysis, automated testing, quality improvement and web-interface integration. The final reported evaluation achieved 80.43% accuracy, 88.24% precision, 78.95% recall and 83.33% F1 score.

The current repository provides the core detection pipeline and Flask web interface. SMS alerting should be described as planned/integration work unless its implementation is added to the repository before final submission.

14. References

- [1] Ultralytics, "YOLO11 Documentation," Ultralytics. <https://docs.ultralytics.com/models/yolo11/>
- [2] Ultralytics, "Object Detection Documentation." <https://docs.ultralytics.com/tasks/detect/>
- [3] Ultralytics, "Track Mode Documentation." <https://docs.ultralytics.com/modes/track/>
- [4] A. Rosebrock, "Intersection over Union (IoU) for Object Detection," PyImageSearch, 2016. <https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/>
- [5] Project repository: <https://github.com/gagan23-ai/highway-accident-detection>